

05 Data Preparation

Course Name: Data Management for Machine Learning
Assignment Title: End-to-End Data Management Pipeline for a Recommendation System
Assignment: Group 51 - Data management for Machine Learning Group 51

Team Members

Team Member Name	Team Member ID
BANSHIDHAR RATH	2025AE05346
JITENDRA KUMAR TIWARI	2025AE05518
KATBA ANKIT CHIMANBHAI	2025AE05229
NAVEEN SURATHU	2025AE05492

1. Objective

This report documents the data preparation stage by capturing project assets, preprocessing evidence, exploratory analysis outputs, and the prepared dataset generated for downstream transformation and modeling.

2. Objective Coverage

- Handle missing user-item interactions and remove invalid or duplicate interaction rows.
- Encode categorical attributes such as product category using one-hot columns where present.
- Normalize numerical variables such as price and parse or standardize timestamps.
- Produce EDA outputs for interaction distributions, item popularity, and sparsity patterns.
- Save a prepared dataset ready for the next pipeline stage.

3. Data Preparation Assets

The following notebooks or scripts were identified as likely data preparation, cleaning, EDA, transformation, or feature-related assets in the project.

Notebook / Script	Relative Path	Last Modified	Size
eda_prep-copy_executed.ipynb	runs\prefect\20260429_115432\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb	2026-04-29 12:03:27	46.45 KB
eda_prep-copy_executed.ipynb	runs\prefect\20260429_121758\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb	2026-04-29 12:23:34	46.44 KB
eda_prep-copy_executed.ipynb	runs\prefect\20260429_132217\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb	2026-04-29 13:28:13	50.87 KB
eda_prep-copy_executed.ipynb	runs\prefect\20260429_160933\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb	2026-04-29 16:16:51	545.98 KB
materialize_features_executed.ipynb	runs\prefect\20260429_160933\executed_notebooks\materialize_features\materialize_features_executed.ipynb	2026-04-29 16:25:43	17.27 KB

Notebook / Script	Relative Path	Last Modified	Size
eda_prep-copy_executed.ipynb	runs\prefect\20260429_164131\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb	2026-04-29 16:51:46	546.07 KB
materialize_features_executed.ipynb	runs\prefect\20260429_164131\executed_notebooks\materialize_features\materialize_features_executed.ipynb	2026-04-29 17:05:20	17.27 KB
eda_prep-copy_executed.ipynb	runs\prefect\20260429T072423Z\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb	2026-04-29 13:18:16	488.47 KB
eda_prep-copy.ipynb	src\03-eda-prep\eda_prep-copy.ipynb	2026-04-29 15:04:17	540.81 KB
materialize_features.ipynb	src\04-features\materialize_features.ipynb	2026-04-29 15:22:54	15.82 KB

4. Supporting Project and Log Artifacts

These supporting files were discovered from raw, bronze, and validation outputs. Long paths are wrapped using real line breaks at safe separators to avoid PDF cut-off issues.

Artifact	Relative Path	Last Modified	Size
events.csv	data\raw\retailrocket\load_date=2026-04-29\load_hour=11\events.csv	2026-04-20 17:28:00	92029.21 KB
products.parquet	data\bronze\dummyjson\products.parquet	2026-04-29 16:42:35	73.89 KB
run_validation.txt	src\02-validation\run_validation.txt	2026-04-28 20:14:12	46.94 KB
data_quality_report_20260429T111342Z.json	reports\validation\data_quality_report_20260429T111342Z.json	2026-04-29 16:47:43	6.66 KB
data_quality_report_20260429T111342Z.pdf	reports\validation\data_quality_report_20260429T111342Z.pdf	2026-04-29 16:47:44	8.49 KB
fix_log_20260429T111342Z.csv	reports\validation\fix_log_20260429T111342Z.csv	2026-04-29 16:46:38	3.43 KB

5. Raw and Prepared Storage Structure

Raw Data Tree

```

raw/
├── dummyjson/
│   ├── load_date=2026-04-29/
│   │   ├── load_hour=11/
│   │   │   ├── categories_raw.json
│   │   │   └── products_raw.json
│   └── retailrocket/
│       ├── load_date=2026-04-29/
│       │   ├── load_hour=11/
│       │   │   ├── category_tree.csv
│       │   │   ├── events.csv
│       │   │   ├── item_properties_part1.csv
│       │   │   └── item_properties_part2.csv

```

Prepared Data Tree

```

prepared/
├── categories_prepared.csv
├── interactions_prepared.csv
├── prepared_interactions_20260429_215146.parquet
├── prepared_interactions_20260429_215229.parquet
├── prepared_interactions_20260429_215532.parquet
└── products_prepared.csv

```

■■■ retailrocket_item_snapshot.csv

features/

6. Preparation Actions Applied

- Standardized common interaction columns where present.
- Dropped rows with missing user_id/item_id where applicable: 0 rows removed.
- Normalized event_type to lowercase string values.
- Parsed event_ts from epoch milliseconds.
- Normalized event timestamp to event_ts_norm.
- Removed duplicate interactions using keys user_id, item_id, event_type, event_ts: 460 rows removed.
- Interaction rows after preparation: 2755641.
- Standardized common product columns where present.
- Encoded category using one-hot columns.
- Filled missing price values with median.
- Normalized price using min-max scaling.
- Missing/invalid price count before imputation: 0.
- Removed duplicate product_id rows: 0.
- Product rows after preparation: 194.

7. Prepared Dataset Inventory

Prepared File	Relative Path	Type	Last Modified	Size
categories_prepared.csv	data\prepared\categories_prepared.csv	.csv	2026-04-29 16:51:43	0.32 KB
interactions_prepared.csv	data\prepared\interactions_prepared.csv	.csv	2026-04-29 16:51:27	196440.63 KB
prepared_interactions_20260429_215146.parquet	data\prepared\prepared_interactions_20260429_215146.parquet	.parquet	2026-04-29 21:51:47	42988.9 KB
prepared_interactions_20260429_215229.parquet	data\prepared\prepared_interactions_20260429_215229.parquet	.parquet	2026-04-29 21:52:30	42988.9 KB
prepared_interactions_20260429_215532.parquet	data\prepared\prepared_interactions_20260429_215532.parquet	.parquet	2026-04-29 21:55:34	65323.8 KB
products_prepared.csv	data\prepared\products_prepared.csv	.csv	2026-04-29 16:51:43	59.06 KB
retailrocket_item_snapshot.csv	data\prepared\retailrocket_item_snapshot.csv	.csv	2026-04-29 16:51:43	144479.37 KB

8. Dataset Quality Summary After Preparation

dataset	rows	columns	missing_cells	duplicate_rows
prepared_interactions	2755641	8	2733184	0
prepared_products	194	57	92	0

9. Sparsity Summary

users	items	observed_pairs	possible_pairs	density	sparsity
1407580	235061	2145179	330867162380	6e-06	0.999994

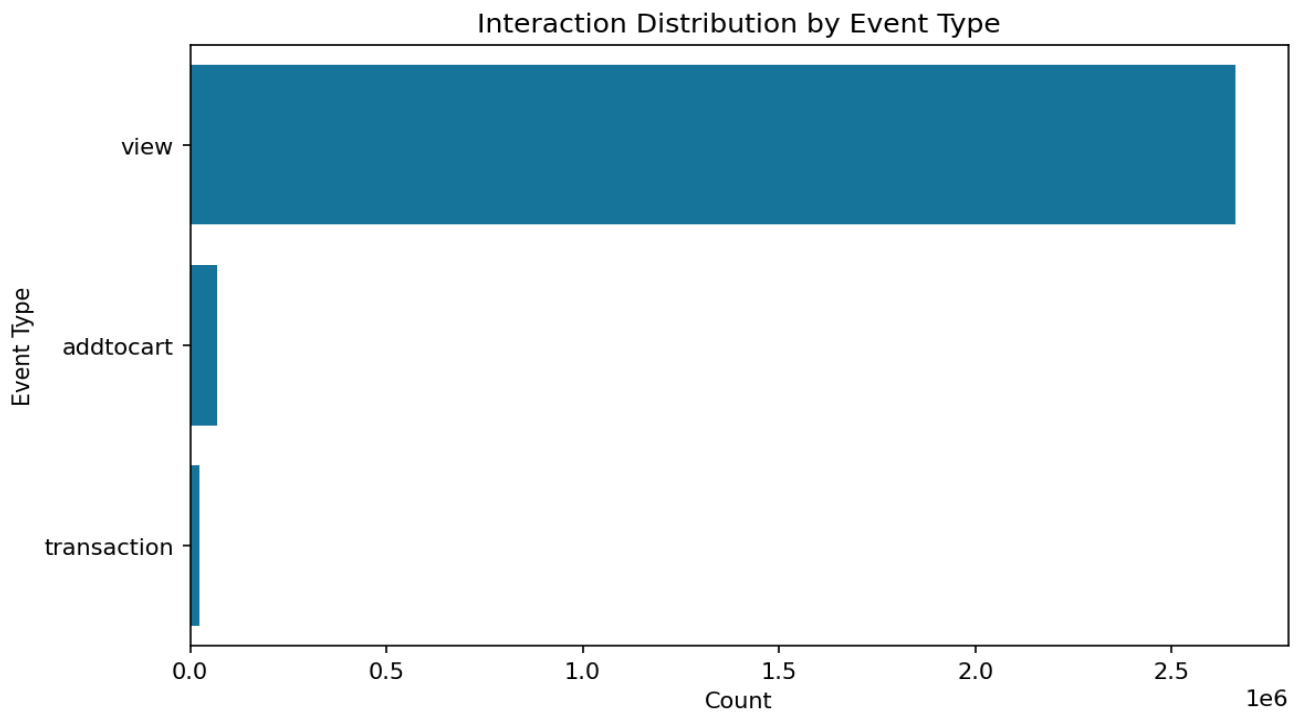
10. Prepared Dataset Preview

event_ts	user_id	event_type	item_id	transactionid	event_hour	event_date	event_ts_norm
2015-06-02 05:02:12.117000	257597	view	355908		5	2015-06-02	0.21800618550646259
2015-06-02 05:50:14.164000	992329	view	248676		5	2015-06-02	0.2182479034187303
2015-06-02 05:13:19.827000	111016	view	318965		5	2015-06-02	0.21806218649020792
2015-06-02 05:12:35.914000	483717	view	253185		5	2015-06-02	0.218058503497293
2015-06-02 05:02:17.106000	951259	view	367447		5	2015-06-02	0.2180066039349838
2015-06-02 05:48:06.234000	972639	view	22556		5	2015-06-02	0.21823717390164873
2015-06-02 05:12:03.240000	810725	view	443030		5	2015-06-02	0.21805576312176636
2015-06-02 05:34:51.897000	794181	view	439202		5	2015-06-02	0.21817055268371682
2015-06-02 04:54:59.221000	824915	view	428805		4	2015-06-02	0.2179698784242571
2015-06-02 05:00:04.592000	339335	view	82389		5	2015-06-02	0.21799548995681958
2015-06-02 05:16:02.373000	176446	view	10572		5	2015-06-02	0.2180758192587809
2015-06-02 05:08:21.252000	929206	view	410676		5	2015-06-02	0.2180371449396519
2015-06-02 05:50:29.496000	15795	view	44872		5	2015-06-02	0.2182491893169238
2015-06-02 05:41:37.356000	598426	view	156489		5	2015-06-02	0.21820455861873111
2015-06-02 05:47:58.165000	223343	view	402625		5	2015-06-02	0.21823649715285381
2015-06-02 05:22:11.378000	57036	view	334662		5	2015-06-02	0.21810676778884178
2015-06-02 05:33:59.808000	1377281	view	251467		5	2015-06-02	0.2181661839678936
2015-06-02 05:33:56.124000	287857	addtocart	5206		5	2015-06-02	0.21816587499000784
2015-06-02 05:50:44.282000	1370216	view	176721		5	2015-06-02	0.2182504294219779
2015-06-02 04:57:58.505000	158090	addtocart	10572		4	2015-06-02	0.21798491501255113

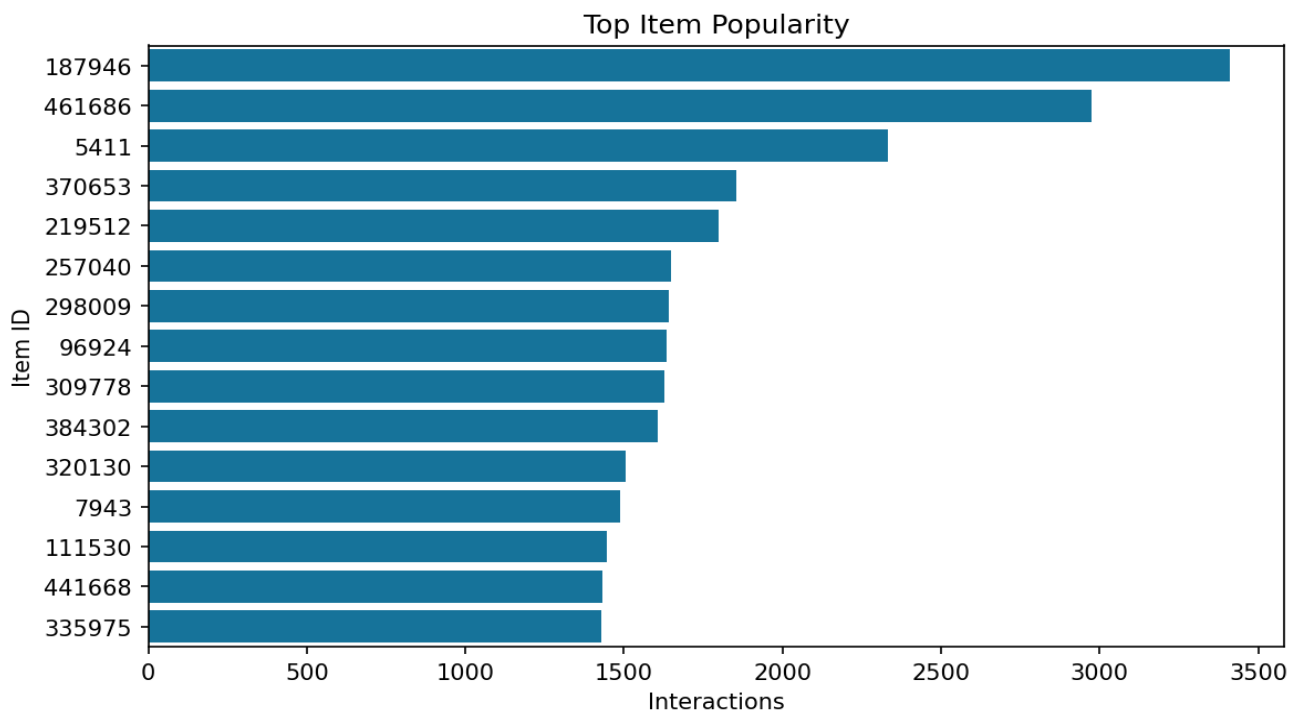
11. Summary Plots

The following plots summarize key EDA views required for the assignment, including histograms and a heatmap when enough interaction data is available.

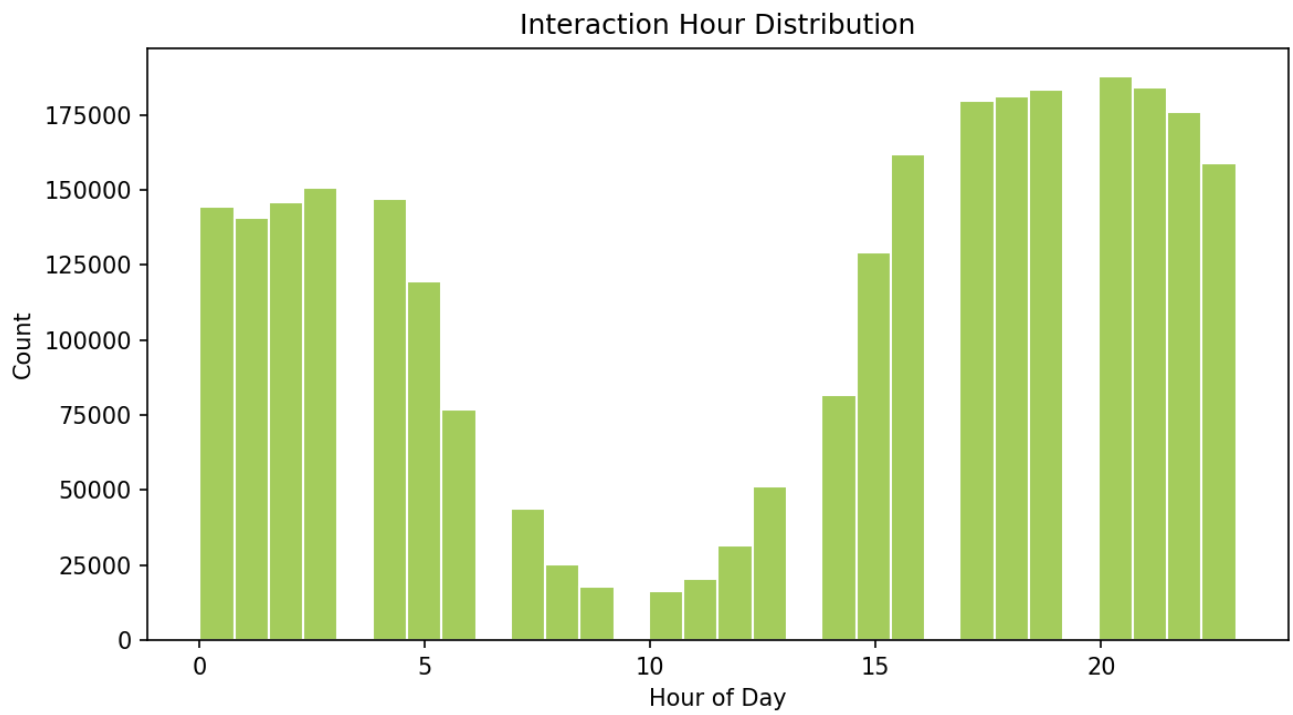
Plot 1: Interaction Distribution



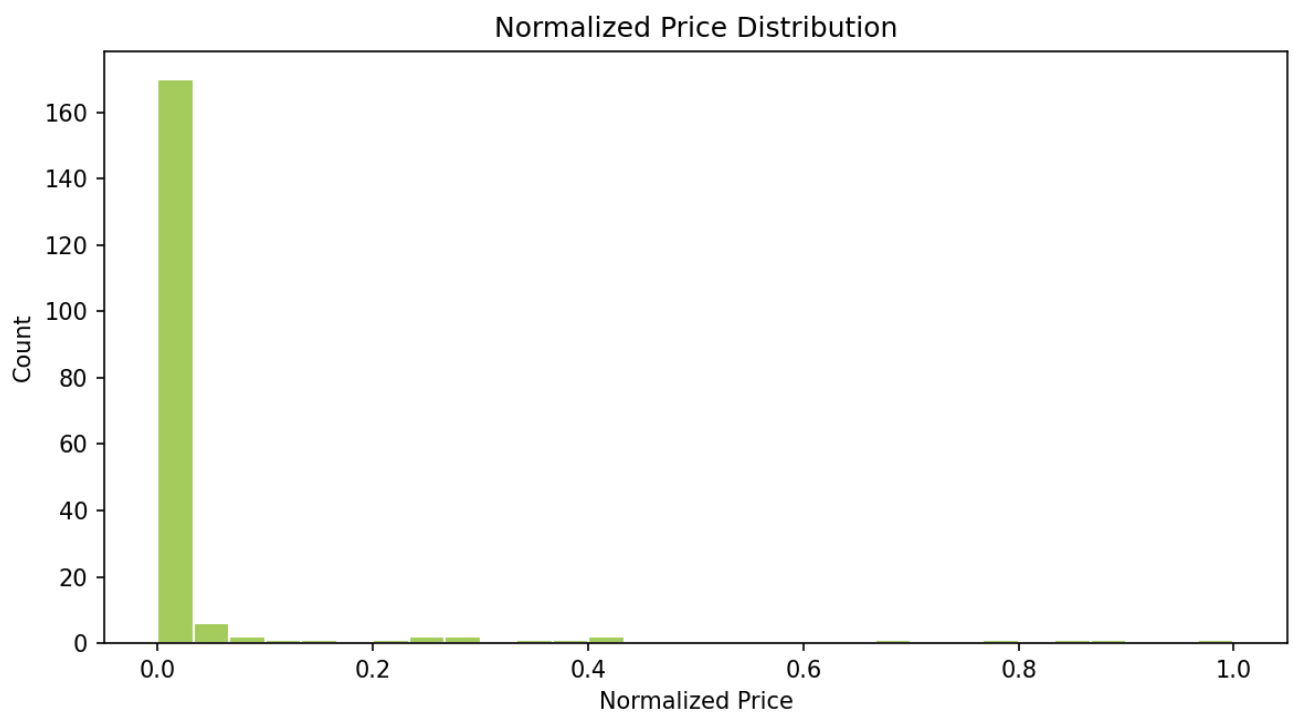
Plot 2: Item Popularity



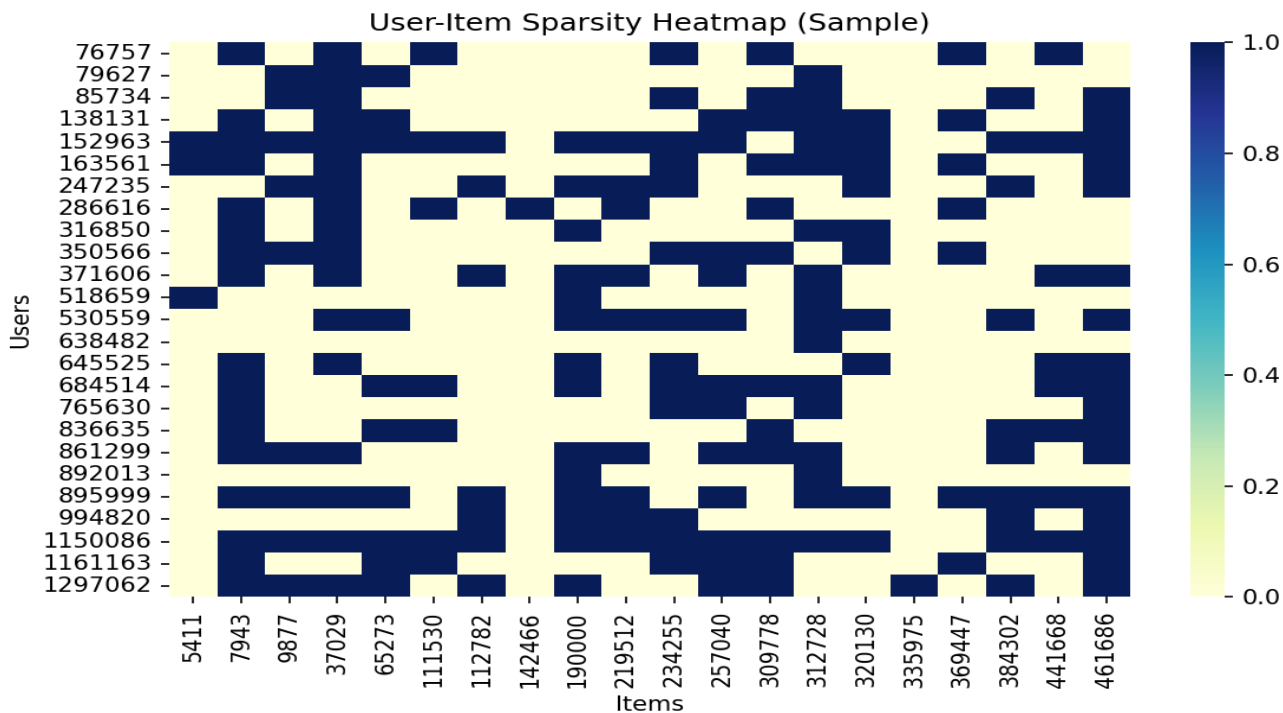
Plot 3: Interaction Hour Hist



Plot 4: Normalized Price Hist



Plot 5: Sparsity Heatmap



12. Data Preparation Code / Notebook Preview

Asset: *eda_prep-copy_executed.ipynb*

Path: runs\prefect\20260429_115432\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb

```
# Code cell 1
import json
import warnings
from pathlib import Path
from datetime import datetime, timezone

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)

try:
    from IPython.display import display
except ImportError:
    def display(x):
        print(x)

# -----
# Project paths and setup
# -----
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports" / "eda"
QUALITY_ROOT = PROJECT_ROOT / "data" / "quality"
QUARANTINE_ROOT = PROJECT_ROOT / "data" / "quarantine"

PREP_ROOT.mkdir(parents=True, exist_ok=True)
REPORT_ROOT.mkdir(parents=True, exist_ok=True)
QUALITY_ROOT.mkdir(parents=True, exist_ok=True)
QUARANTINE_ROOT.mkdir(parents=True, exist_ok=True)

RUN_TS = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")

print("PROJECT_ROOT:", PROJECT_ROOT)
print("RAW_ROOT:", RAW_ROOT)
print("PREP_ROOT:", PREP_ROOT)
print("REPORT_ROOT:", REPORT_ROOT)
print("QUALITY_ROOT:", QUALITY_ROOT)
print("QUARANTINE_ROOT:", QUARANTINE_ROOT)

# -----
# File discovery helpers
# -----
def latest_match(base_dir: Path, pattern: str):
    if not base_dir.exists():
        return None
    matches = list(base_dir.rglob(pattern))
    if not matches:
        return None
    return max(matches, key=lambda p: p.stat().st_mtime)

files = {
    "events_csv": latest_match(RAW_ROOT, "events.csv"),
    "category_tree_csv": latest_match(RAW_ROOT, "category_tree.csv"),
    "item_properties_part1_csv": latest_match(RAW_ROOT, "item_properties_part1.csv"),
    "item_properties_part2_csv": latest_match(RAW_ROOT, "item_properties_part2.csv"),
    "products_raw_json": latest_match(RAW_ROOT, "products_raw.json"),
    "categories_raw_json": latest_match(RAW_ROOT, "categories_raw.json"),
}

for name, path in files.items():
    print(f"{name}: {path}")

missing = [k for k, v in files.items() if v is None]
if missing:
    raise FileNotFoundError(f"Missing required raw files: {missing}")
```

```

# -----
# Validation helpers
# -----
def make_issue(dataset, check_type, severity, issue_count, description, file_path):
    if int(issue_count) <= 0:
        return None
    return {
        "dataset": dataset,
        "check_type": check_type,
        "severity": severity,
        "issue_count": int(issue_count),
        "description": description,
        "file_path": str(file_path),
    }

def summarize_validation(dataset, df, issues):
    errors = sum(i["issue_count"] for i in issues if i["severity"] == "error")
    warnings = sum(i["issue_count"] for i in issues if i["severity"] == "warning")
    return {
        "dataset": dataset,
        "rows": int(len(df)),
        "columns": int(len(df.columns)),
        "errors": int(errors),
        "warnings": int(warnings),
        "status": "FAIL" if errors > 0 else "PASS",
    }

def safe_nunique(series):
    def make_hashable(x):
        if isinstance(x, (list, dict)):
            return json.dumps(x, sort_keys=True)
        return x
    return series.map(make_hashable).nunique(dropna=True)

def profile_df(df, name):
    summary = pd.DataFrame({
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "null_count": df.isna().sum().values,
        "null_pct": (df.isna().mean().values * 100).round(2),
        "nunique": [safe_nunique(df[col]) for col in df.columns]
    })
    print(f"\n{name} shape: {df.shape}")
    display(summary.sort_values(["null_pct", "nunique"], ascending=[False, False]))
    display(df.head())

def save_validation_reports(issues, summaries, prefix):
    issues_df = pd.DataFrame(issues)
    summary_df = pd.DataFrame(summaries)

    issues_path = QUALITY_ROOT / f"{prefix}_issues_{RUN_TS}.csv"
    summary_path = QUALITY_ROOT / f"{prefix}_summary_{RUN_TS}.csv"

    issues_df.to_csv(issues_path, index=False)
    summary_df.to_csv(summary_path, index=False)

    print(f"Saved: {issues_path}")
    print(f"Saved: {summary_path}")

    return issues_df, summary_df, issues_path, summary_path

def safe_to_csv(df, path):
    path.parent.mkdir(parents=True, exist_ok=True)

    if path.exists() and path.is_dir():
        raise IsADirectoryError(f"Expected file but found directory: {path}")

    try:
        df.to_csv(path, index=False)
        print(f"Saved: {path}")
        return path
    except PermissionError:
        fallback = path.with_name(f"{path.stem}_{RUN_TS}{path.suffix}")
        df.to_csv(fallback, index=False)
        print(f"File locked, saved fallback instead: {fallback}")
        return fallback

# -----
# Raw validation functions
# -----
def validate_raw_events(df, file_path):
    issues = []
    d = df.copy()

```

```

d.columns = [c.strip().lower() for c in d.columns]

required = ["timestamp", "visitorid", "event", "itemid"]
missing_cols = [c for c in required if c not in d.columns]
item = make_issue("retailrocket_events", "schema", "error", len(missing_cols), f"Missing
columns: {missing_cols}", file_path)
if item:
    issues.append(item)
    return issues

if "transactionid" not in d.columns:
    d["transactionid"] = np.nan

item = make_issue(
    "retailrocket_events",
    "missing_values",
    "warning",
    d["transactionid"].isna().sum(),
    "Nulls in transactionid",
    file_path
)
if item:
    issues.append(item)

item = make_issue(
    "retailrocket_events",
    "duplicates",
    "warning",
    d.duplicated().sum(),
    "Duplicate rows",
    file_path
)
if item:
    issues.append(item)

return issues

def validate_raw_category_tree(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["categoryid", "parentid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_category_tree", "schema", "error", len(missing_cols),
f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    item = make_issue(
        "retailrocket_category_tree",
        "missing_values",
        "warning",
        d["parentid"].isna().sum(),
        "Nulls in parentid",
        file_path
    )
    if item:
        issues.append(item)

    return issues

def validate_raw_products(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["id", "title",

```

Asset: *eda_prep-copy_executed.ipynb*

Path: runs\prefect\20260429_121758\executed_notebooks\eda_prepleda_prep-copy_executed.ipynb

```

# Code cell 1
import json
import warnings
from pathlib import Path
from datetime import datetime, timezone

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
warnings.filterwarnings("ignore")

```

```

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)

try:
    from IPython.display import display
except ImportError:
    def display(x):
        print(x)

# -----
# Project paths and setup
# -----
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports" / "eda"
QUALITY_ROOT = PROJECT_ROOT / "data" / "quality"
QUARANTINE_ROOT = PROJECT_ROOT / "data" / "quarantine"

PREP_ROOT.mkdir(parents=True, exist_ok=True)
REPORT_ROOT.mkdir(parents=True, exist_ok=True)
QUALITY_ROOT.mkdir(parents=True, exist_ok=True)
QUARANTINE_ROOT.mkdir(parents=True, exist_ok=True)

RUN_TS = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")

print("PROJECT_ROOT:", PROJECT_ROOT)
print("RAW_ROOT:", RAW_ROOT)
print("PREP_ROOT:", PREP_ROOT)
print("REPORT_ROOT:", REPORT_ROOT)
print("QUALITY_ROOT:", QUALITY_ROOT)
print("QUARANTINE_ROOT:", QUARANTINE_ROOT)

# -----
# File discovery helpers
# -----
def latest_match(base_dir: Path, pattern: str):
    if not base_dir.exists():
        return None
    matches = list(base_dir.rglob(pattern))
    if not matches:
        return None
    return max(matches, key=lambda p: p.stat().st_mtime)

files = {
    "events_csv": latest_match(RAW_ROOT, "events.csv"),
    "category_tree_csv": latest_match(RAW_ROOT, "category_tree.csv"),
    "item_properties_part1_csv": latest_match(RAW_ROOT, "item_properties_part1.csv"),
    "item_properties_part2_csv": latest_match(RAW_ROOT, "item_properties_part2.csv"),
    "products_raw_json": latest_match(RAW_ROOT, "products_raw.json"),
    "categories_raw_json": latest_match(RAW_ROOT, "categories_raw.json"),
}

for name, path in files.items():
    print(f"{name}: {path}")

missing = [k for k, v in files.items() if v is None]
if missing:
    raise FileNotFoundError(f"Missing required raw files: {missing}")

# -----
# Validation helpers
# -----
def make_issue(dataset, check_type, severity, issue_count, description, file_path):
    if int(issue_count) <= 0:
        return None
    return {
        "dataset": dataset,
        "check_type": check_type,
        "severity": severity,
        "issue_count": int(issue_count),
        "description": description,
        "file_path": str(file_path),
    }

def summarize_validation(dataset, df, issues):
    errors = sum(i["issue_count"] for i in issues if i["severity"] == "error")
    warnings = sum(i["issue_count"] for i in issues if i["severity"] == "warning")

```

```

    return {
        "dataset": dataset,
        "rows": int(len(df)),
        "columns": int(len(df.columns)),
        "errors": int(errors),
        "warnings": int(warnings),
        "status": "FAIL" if errors > 0 else "PASS",
    }

def safe_nunique(series):
    def make_hashable(x):
        if isinstance(x, (list, dict)):
            return json.dumps(x, sort_keys=True)
        return x
    return series.map(make_hashable).nunique(dropna=True)

def profile_df(df, name):
    summary = pd.DataFrame({
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "null_count": df.isna().sum().values,
        "null_pct": (df.isna().mean().values * 100).round(2),
        "nunique": [safe_nunique(df[col]) for col in df.columns]
    })
    print(f"\n{name} shape: {df.shape}")
    display(summary.sort_values(["null_pct", "nunique"], ascending=[False, False]))
    display(df.head())

def save_validation_reports(issues, summaries, prefix):
    issues_df = pd.DataFrame(issues)
    summary_df = pd.DataFrame(summaries)

    issues_path = QUALITY_ROOT / f"{prefix}_issues_{RUN_TS}.csv"
    summary_path = QUALITY_ROOT / f"{prefix}_summary_{RUN_TS}.csv"

    issues_df.to_csv(issues_path, index=False)
    summary_df.to_csv(summary_path, index=False)

    print(f"Saved: {issues_path}")
    print(f"Saved: {summary_path}")

    return issues_df, summary_df, issues_path, summary_path

def safe_to_csv(df, path):
    path.parent.mkdir(parents=True, exist_ok=True)

    if path.exists() and path.is_dir():
        raise IsADirectoryError(f"Expected file but found directory: {path}")

    try:
        df.to_csv(path, index=False)
        print(f"Saved: {path}")
        return path
    except PermissionError:
        fallback = path.with_name(f"{path.stem}_{RUN_TS}{path.suffix}")
        df.to_csv(fallback, index=False)
        print(f"File locked, saved fallback instead: {fallback}")
        return fallback

# -----
# Raw validation functions
# -----
def validate_raw_events(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["timestamp", "visitorid", "event", "itemid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_events", "schema", "error", len(missing_cols), f"Missing
columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    if "transactionid" not in d.columns:
        d["transactionid"] = np.nan

    item = make_issue(
        "retailrocket_events",
        "missing_values",
        "warning",
        d["transactionid"].isna().sum(),
        "Nulls in transactionid",
    )

```

```

        file_path
    )
    if item:
        issues.append(item)

    item = make_issue(
        "retailrocket_events",
        "duplicates",
        "warning",
        d.duplicated().sum(),
        "Duplicate rows",
        file_path
    )
    if item:
        issues.append(item)

    return issues

def validate_raw_category_tree(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["categoryid", "parentid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_category_tree", "schema", "error", len(missing_cols),
f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    item = make_issue(
        "retailrocket_category_tree",
        "missing_values",
        "warning",
        d["parentid"].isna().sum(),
        "Nulls in parentid",
        file_path
    )
    if item:
        issues.append(item)

    return issues

def validate_raw_products(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["id", "title",

```

Asset: eda_prep-copy_executed.ipynb

Path: runs\prefect\20260429_132217\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb

```

# Code cell 1
import sys
print(sys.executable)
print(sys.version)
import sys
!"{sys.executable}" -m pip uninstall -y matplotlib seaborn
!"{sys.executable}" -m pip install --no-cache-dir --upgrade pip setuptools wheel
!"{sys.executable}" -m pip install --no-cache-dir matplotlib seaborn
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
print(matplotlib.__version__)
print("imports ok")

import json
import warnings
from pathlib import Path
from datetime import datetime, timezone

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)

```

```

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)

try:
    from IPython.display import display
except ImportError:
    def display(x):
        print(x)

# -----
# Project paths and setup
# -----
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports" / "eda"
QUALITY_ROOT = PROJECT_ROOT / "data" / "quality"
QUARANTINE_ROOT = PROJECT_ROOT / "data" / "quarantine"

PREP_ROOT.mkdir(parents=True, exist_ok=True)
REPORT_ROOT.mkdir(parents=True, exist_ok=True)
QUALITY_ROOT.mkdir(parents=True, exist_ok=True)
QUARANTINE_ROOT.mkdir(parents=True, exist_ok=True)

RUN_TS = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")

print("PROJECT_ROOT:", PROJECT_ROOT)
print("RAW_ROOT:", RAW_ROOT)
print("PREP_ROOT:", PREP_ROOT)
print("REPORT_ROOT:", REPORT_ROOT)
print("QUALITY_ROOT:", QUALITY_ROOT)
print("QUARANTINE_ROOT:", QUARANTINE_ROOT)

# -----
# File discovery helpers
# -----
def latest_match(base_dir: Path, pattern: str):
    if not base_dir.exists():
        return None
    matches = list(base_dir.rglob(pattern))
    if not matches:
        return None
    return max(matches, key=lambda p: p.stat().st_mtime)

files = {
    "events_csv": latest_match(RAW_ROOT, "events.csv"),
    "category_tree_csv": latest_match(RAW_ROOT, "category_tree.csv"),
    "item_properties_part1_csv": latest_match(RAW_ROOT, "item_properties_part1.csv"),
    "item_properties_part2_csv": latest_match(RAW_ROOT, "item_properties_part2.csv"),
    "products_raw_json": latest_match(RAW_ROOT, "products_raw.json"),
    "categories_raw_json": latest_match(RAW_ROOT, "categories_raw.json"),
}

for name, path in files.items():
    print(f"{name}: {path}")

missing = [k for k, v in files.items() if v is None]
if missing:
    raise FileNotFoundError(f"Missing required raw files: {missing}")

# -----
# Validation helpers
# -----
def make_issue(dataset, check_type, severity, issue_count, description, file_path):
    if int(issue_count) <= 0:
        return None
    return {
        "dataset": dataset,
        "check_type": check_type,
        "severity": severity,
        "issue_count": int(issue_count),
        "description": description,
        "file_path": str(file_path),
    }

def summarize_validation(dataset, df, issues):
    errors = sum(i["issue_count"] for i in issues if i["severity"] == "error")
    warnings = sum(i["issue_count"] for i in issues if i["severity"] == "warning")
    return {
        "dataset": dataset,
        "rows": int(len(df)),
    }

```

```

        "columns": int(len(df.columns)),
        "errors": int(errors),
        "warnings": int(warnings),
        "status": "FAIL" if errors > 0 else "PASS",
    }

def safe_nunique(series):
    def make_hashable(x):
        if isinstance(x, (list, dict)):
            return json.dumps(x, sort_keys=True)
        return x
    return series.map(make_hashable).nunique(dropna=True)

def profile_df(df, name):
    summary = pd.DataFrame({
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "null_count": df.isna().sum().values,
        "null_pct": (df.isna().mean().values * 100).round(2),
        "nunique": [safe_nunique(df[col]) for col in df.columns]
    })
    print(f"\n{name} shape: {df.shape}")
    display(summary.sort_values(["null_pct", "nunique"], ascending=[False, False]))
    display(df.head())

def save_validation_reports(issues, summaries, prefix):
    issues_df = pd.DataFrame(issues)
    summary_df = pd.DataFrame(summaries)

    issues_path = QUALITY_ROOT / f"{prefix}_issues_{RUN_TS}.csv"
    summary_path = QUALITY_ROOT / f"{prefix}_summary_{RUN_TS}.csv"

    issues_df.to_csv(issues_path, index=False)
    summary_df.to_csv(summary_path, index=False)

    print(f"Saved: {issues_path}")
    print(f"Saved: {summary_path}")

    return issues_df, summary_df, issues_path, summary_path

def safe_to_csv(df, path):
    path.parent.mkdir(parents=True, exist_ok=True)

    if path.exists() and path.is_dir():
        raise IsADirectoryError(f"Expected file but found directory: {path}")

    try:
        df.to_csv(path, index=False)
        print(f"Saved: {path}")
        return path
    except PermissionError:
        fallback = path.with_name(f"{path.stem}_{RUN_TS}{path.suffix}")
        df.to_csv(fallback, index=False)
        print(f"File locked, saved fallback instead: {fallback}")
        return fallback

# -----
# Raw validation functions
# -----
def validate_raw_events(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["timestamp", "visitorid", "event", "itemid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_events", "schema", "error", len(missing_cols), f"Missing
columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    if "transactionid" not in d.columns:
        d["transactionid"] = np.nan

    item = make_issue(
        "retailrocket_events",
        "missing_values",
        "warning",
        d["transactionid"].isna().sum(),
        "Nulls in transactionid",
        file_path
    )
    if item:

```

```

        issues.append(item)

    item = make_issue(
        "retailrocket_events",
        "duplicates",
        "warning",
        d.duplicated().sum(),
        "Duplicate rows",
        file_path
    )
    if item:
        issues.append(item)

    return issues

def validate_raw_category_tree(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["categoryid", "parentid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_category_tree", "schema", "error", len(missing_cols),
f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
    return issues

    item = make_issue(

```

Asset: *eda_prep-copy_executed.ipynb*

Path: runs\prefect\20260429_160933\executed_notebooks\eda_prep\eda_prep-copy_executed.ipynb

```

# Code cell 1
import json
import warnings
from pathlib import Path
from datetime import datetime, timezone

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)

try:
    from IPython.display import display
except ImportError:
    def display(x):
        print(x)

# -----
# Project paths and setup
# -----
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
REPORT_ROOT = PROJECT_ROOT / "reports" / "eda"
QUALITY_ROOT = PROJECT_ROOT / "data" / "quality"
QUARANTINE_ROOT = PROJECT_ROOT / "data" / "quarantine"

PREP_ROOT.mkdir(parents=True, exist_ok=True)
REPORT_ROOT.mkdir(parents=True, exist_ok=True)
QUALITY_ROOT.mkdir(parents=True, exist_ok=True)
QUARANTINE_ROOT.mkdir(parents=True, exist_ok=True)

RUN_TS = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")

print("PROJECT_ROOT:", PROJECT_ROOT)
print("RAW_ROOT:", RAW_ROOT)
print("PREP_ROOT:", PREP_ROOT)
print("REPORT_ROOT:", REPORT_ROOT)
print("QUALITY_ROOT:", QUALITY_ROOT)
print("QUARANTINE_ROOT:", QUARANTINE_ROOT)

```

```

# -----
# File discovery helpers
# -----
def latest_match(base_dir: Path, pattern: str):
    if not base_dir.exists():
        return None
    matches = list(base_dir.rglob(pattern))
    if not matches:
        return None
    return max(matches, key=lambda p: p.stat().st_mtime)

files = {
    "events_csv": latest_match(RAW_ROOT, "events.csv"),
    "category_tree_csv": latest_match(RAW_ROOT, "category_tree.csv"),
    "item_properties_part1_csv": latest_match(RAW_ROOT, "item_properties_part1.csv"),
    "item_properties_part2_csv": latest_match(RAW_ROOT, "item_properties_part2.csv"),
    "products_raw_json": latest_match(RAW_ROOT, "products_raw.json"),
    "categories_raw_json": latest_match(RAW_ROOT, "categories_raw.json"),
}

for name, path in files.items():
    print(f"{name}: {path}")

missing = [k for k, v in files.items() if v is None]
if missing:
    raise FileNotFoundError(f"Missing required raw files: {missing}")

# -----
# Validation helpers
# -----
def make_issue(dataset, check_type, severity, issue_count, description, file_path):
    if int(issue_count) <= 0:
        return None
    return {
        "dataset": dataset,
        "check_type": check_type,
        "severity": severity,
        "issue_count": int(issue_count),
        "description": description,
        "file_path": str(file_path),
    }

def summarize_validation(dataset, df, issues):
    errors = sum(i["issue_count"] for i in issues if i["severity"] == "error")
    warnings = sum(i["issue_count"] for i in issues if i["severity"] == "warning")
    return {
        "dataset": dataset,
        "rows": int(len(df)),
        "columns": int(len(df.columns)),
        "errors": int(errors),
        "warnings": int(warnings),
        "status": "FAIL" if errors > 0 else "PASS",
    }

def safe_nunique(series):
    def make_hashable(x):
        if isinstance(x, (list, dict)):
            return json.dumps(x, sort_keys=True)
        return x
    return series.map(make_hashable).nunique(dropna=True)

def profile_df(df, name):
    summary = pd.DataFrame({
        "column": df.columns,
        "dtype": df.dtypes.astype(str).values,
        "null_count": df.isna().sum().values,
        "null_pct": (df.isna().mean().values * 100).round(2),
        "nunique": [safe_nunique(df[col]) for col in df.columns]
    })
    print(f"\n{name} shape: {df.shape}")
    display(summary.sort_values(["null_pct", "nunique"], ascending=[False, False]))
    display(df.head())

def save_validation_reports(issues, summaries, prefix):
    issues_df = pd.DataFrame(issues)
    summary_df = pd.DataFrame(summaries)

    issues_path = QUALITY_ROOT / f"{prefix}_issues_{RUN_TS}.csv"
    summary_path = QUALITY_ROOT / f"{prefix}_summary_{RUN_TS}.csv"

    issues_df.to_csv(issues_path, index=False)
    summary_df.to_csv(summary_path, index=False)
    print(f"Saved: {issues_path}")

```

```

print(f"Saved: {summary_path}")

return issues_df, summary_df, issues_path, summary_path

def safe_to_csv(df, path):
    path.parent.mkdir(parents=True, exist_ok=True)

    if path.exists() and path.is_dir():
        raise IsADirectoryError(f"Expected file but found directory: {path}")

    try:
        df.to_csv(path, index=False)
        print(f"Saved: {path}")
        return path
    except PermissionError:
        fallback = path.with_name(f"{path.stem}_{RUN_TS}{path.suffix}")
        df.to_csv(fallback, index=False)
        print(f"File locked, saved fallback instead: {fallback}")
        return fallback

# -----
# Raw validation functions
# -----
def validate_raw_events(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["timestamp", "visitorid", "event", "itemid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_events", "schema", "error", len(missing_cols), f"Missing
columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    if "transactionid" not in d.columns:
        d["transactionid"] = np.nan

    item = make_issue(
        "retailrocket_events",
        "missing_values",
        "warning",
        d["transactionid"].isna().sum(),
        "Nulls in transactionid",
        file_path
    )
    if item:
        issues.append(item)

    item = make_issue(
        "retailrocket_events",
        "duplicates",
        "warning",
        d.duplicated().sum(),
        "Duplicate rows",
        file_path
    )
    if item:
        issues.append(item)

    return issues

def validate_raw_category_tree(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["categoryid", "parentid"]
    missing_cols = [c for c in required if c not in d.columns]
    item = make_issue("retailrocket_category_tree", "schema", "error", len(missing_cols),
f"Missing columns: {missing_cols}", file_path)
    if item:
        issues.append(item)
        return issues

    item = make_issue(
        "retailrocket_category_tree",
        "missing_values",
        "warning",
        d["parentid"].isna().sum(),
        "Nulls in parentid",
        file_path
    )
    if item:
        issues.append(item)

```

```

        return issues

def validate_raw_products(df, file_path):
    issues = []
    d = df.copy()
    d.columns = [c.strip().lower() for c in d.columns]

    required = ["id", "title",

```

Asset: materialize_features_executed.ipynb

Path:

runs\prefect\20260429_160933\executed_notebooks\materialize_features\materialize_features_executed.ipynb

```

# Code cell 1
from __future__ import annotations

from pathlib import Path
import numpy as np
import pandas as pd

# -----
# Project paths and setup
# -----
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
#PROJECT_ROOT = Path(__file__).resolve().parents[1]
PREP_ROOT = PROJECT_ROOT / "data" / "prepared"
FEATURE_ROOT = PROJECT_ROOT / "data" / "featured" / "v1"

INTERACTIONS_PATH = PREP_ROOT / "interactions_prepared.csv"
PRODUCTS_PATH = PREP_ROOT / "products_prepared.csv"

USER_FEATURES_PATH = FEATURE_ROOT / "user_features.parquet"
ITEM_FEATURES_PATH = FEATURE_ROOT / "item_features.parquet"
INTERACTION_FEATURES_PATH = FEATURE_ROOT / "interaction_features.parquet"

def min_max_scale(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    if s.notna().sum() == 0:
        return pd.Series(np.nan, index=series.index)

    min_val = s.min()
    max_val = s.max()

    if pd.isna(min_val) or pd.isna(max_val):
        return pd.Series(np.nan, index=series.index)

    if min_val == max_val:
        return pd.Series(0.0, index=series.index)

    return (s - min_val) / (max_val - min_val)

def ensure_required_columns(df: pd.DataFrame, required: list[str], df_name: str) -> None:
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"{df_name} is missing required columns: {missing}")

def load_prepared_data() -> tuple[pd.DataFrame, pd.DataFrame]:
    if not INTERACTIONS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {INTERACTIONS_PATH}")

    if not PRODUCTS_PATH.exists():
        raise FileNotFoundError(f"Missing file: {PRODUCTS_PATH}")

    interactions = pd.read_csv(INTERACTIONS_PATH)
    products = pd.read_csv(PRODUCTS_PATH)

    ensure_required_columns(
        interactions,
        ["user_id", "item_id", "event", "event_weight", "event_ts", "event_date"],
        "interactions_prepared.csv",
    )
    ensure_required_columns(
        products,
        ["category"],
        "products_prepared.csv",
    )

    interactions["user_id"] = pd.to_numeric(interactions["user_id"],
errors="coerce").astype("Int64")

```

```

        interactions["item_id"] = pd.to_numeric(interactions["item_id"],
errors="coerce").astype("Int64")
        interactions["event"] = interactions["event"].astype(str).str.strip().str.lower()
        interactions["event_weight"] = pd.to_numeric(interactions["event_weight"], errors="coerce")
        interactions["event_ts"] = pd.to_datetime(interactions["event_ts"], errors="coerce")
        interactions["event_date"] = pd.to_datetime(interactions["event_date"], errors="coerce")

        interactions = interactions.dropna(subset=["user_id", "item_id", "event", "event_weight",
"event_ts"])
        interactions["user_id"] = interactions["user_id"].astype("int64")
        interactions["item_id"] = interactions["item_id"].astype("int64")

        products.columns = [c.strip() for c in products.columns]

        return interactions, products

def normalize_products(products: pd.DataFrame) -> pd.DataFrame:
    products = products.copy()

    if "item_id" not in products.columns:
        if "product_id" in products.columns:
            products["item_id"] = pd.to_numeric(products["product_id"], errors="coerce")
        elif "id" in products.columns:
            products["item_id"] = pd.to_numeric(products["id"], errors="coerce")

    if "item_id" not in products.columns:
        raise ValueError("products_prepared.csv must contain one of: item_id, product_id, or
id")

    products["item_id"] = pd.to_numeric(products["item_id"], errors="coerce")
    products = products.dropna(subset=["item_id"]).copy()
    products["item_id"] = products["item_id"].astype("int64")

    if "category" in products.columns:
        products["category"] = (
            products["category"]
            .astype(str)
            .str.strip()
            .str.lower()
        )
    else:
        products["category"] = "unknown"

    if "price_norm" not in products.columns:
        products["price_norm"] = min_max_scale(products["price"]) if "price" in
products.columns else 0.0

    if "rating_norm" not in products.columns:
        if "rating" in products.columns:
            products["rating_norm"] = min_max_scale(products["rating"])
        else:
            products["rating_norm"] = 0.0

    if "stock_norm" not in products.columns:
        if "stock" in products.columns:
            products["stock_norm"] = min_max_scale(products["stock"])
        else:
            products["stock_norm"] = 0.0

    keep_cols = ["item_id", "price_norm", "rating_norm", "stock_norm", "category"]
    return products[keep_cols].drop_duplicates(subset=["item_id"]).reset_index(drop=True)

def build_user_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    user_features = (
        interactions.groupby("user_id")
        .agg(
            user_total_interactions=("user_id", "size"),
            user_view_count=("event", lambda s: (s == "view").sum()),
            user_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
            user_transaction_count=("event", lambda s: (s == "transaction").sum()),
            user_avg_event_weight=("event_weight", "mean"),
            user_active_days=("event_date", lambda s: s.dt.date.nunique()),
            last_event_ts=("event_ts", "max"),
        )
        .reset_index()
    )

    user_features["user_last_seen_recency_days"] = (
        (max_ts - user_features["last_event_ts"]).dt.total_seconds() / 86400.0
    )

    user_features = user_features.drop(columns=["last_event_ts"])

    return user_features

def build_item_features(interactions: pd.DataFrame, products: pd.DataFrame) -> pd.DataFrame:

```

```

item_features = (
    interactions.groupby("item_id")
    .agg(
        item_total_interactions=("item_id", "size"),
        item_view_count=("event", lambda s: (s == "view").sum()),
        item_addtocart_count=("event", lambda s: (s == "addtocart").sum()),
        item_transaction_count=("event", lambda s: (s == "transaction").sum()),
        item_avg_event_weight=("event_weight", "mean"),
    )
    .reset_index()
)

item_features["item_popularity_rank"] = (
    item_features["item_total_interactions"]
    .rank(method="dense", ascending=False)
    .astype("int64")
)

products_norm = normalize_products(products)

item_features = item_features.merge(products_norm, on="item_id", how="left")

item_features["category"] = item_features["category"].fillna("unknown")
for col in ["price_norm", "rating_norm", "stock_norm"]:
    item_features[col] = item_features[col].fillna(0.0)

return item_features

def build_interaction_features(interactions: pd.DataFrame) -> pd.DataFrame:
    max_ts = interactions["event_ts"].max()

    interaction_features = (

```

13. Validation / Fix Log Preview

```

{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": 2,
      "id": "fef32579-f890-4b53-8d24-2fcae64a4914",
      "metadata": {},
      "outputs": [
        {
          "name": "stdout",
          "output_type": "stream",
          "text": [
            "PROJECT_ROOT: C:\\Users\\barath\\recomart-pipeline\\n",
            "RAW_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\n",
            "BRONZE_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\bronze\\n",
            "\\n",
            "Discovered files:\\n",
            "- events_csv: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\data\\raw\\retailrocket\\load_date=2026-04-28\\load_hour=14\\events.csv\\n",
            "- category_tree_csv: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\data\\raw\\retailrocket\\load_date=2026-04-26\\load_hour=19\\category_tree.csv\\n",
            "- item_properties_part1_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock\\n",
            "et\\load_date=2026-04-28\\load_hour=14\\item_properties_part1.csv\\n",
            "- item_properties_part2_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock\\n",
            "et\\load_date=2026-04-28\\load_hour=14\\item_properties_part2.csv\\n",
            "- products_raw_json: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\products_raw.json\\n",
            "- categories_raw_json: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\categories_raw.json\\n",
            "- products_parquet: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\data\\bronze\\dummyjson\\products.parquet\\n",
            "- categories_parquet: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\data\\bronze\\dummyjson\\categories.parquet\\n",
            "\\n",
            "Initial status: FAIL\\n",
            "Final status: PASS\\n",
            "Initial issues file: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\reports\\validation\\validation_issues_initial_20260428T144031Z.csv\\n",
            "Initial summary file: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\reports\\validation\\validation_summary_initial_20260428T144031Z.csv\\n",
            "Revalidation issues file: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\reports\\validation\\validation_issues_revalidated_20260428T144031Z.csv\\n",
            "Revalidation summary file: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\reports\\validation\\validation_summary_revalidated_20260428T144031Z.csv\\n",
            "Fix log file: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\reports\\validation\\fix_log_20260428T144031Z.csv\\n",
            "JSON report file: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.json\\n",
            "PDF report file: C:\\Users\\barath\\recomart-\\n",
            "pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.pdf\\n"
          ]
        }
      ]
    }
  ]
}

```

```

    ]
  }
],
"source": [
  "## DM4ML - Assignment - Validation + Auto-Fix + Revalidation + PDF Report\n",
  "\n",
  "import json\n",
  "from pathlib import Path\n",
  "from datetime import datetime, timezone\n",
  "\n",
  "import pandas as pd\n",
  "\n",
  "from reportlab.lib import colors\n",
  "from reportlab.lib.pagesizes import A4\n",
  "from reportlab.lib.styles import getSampleStyleSheet\n",
  "from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle,
PageBreak\n",
  "\n",
  "## =====\n",
  "## RecoMart Validation - robust file discovery + validation\n",
  "## with remediation, revalidation, and PDF reporting\n",
  "## =====\n",
  "\n",
  "PROJECT_ROOT = Path.cwd().resolve()\n",
  "while not (PROJECT_ROOT / \"src\").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:\n",
  "    PROJECT_ROOT = PROJECT_ROOT.parent\n",
  "\n",
  "RAW_ROOT = PROJECT_ROOT / \"data\" / \"raw\"\n",
  "BRONZE_ROOT = PROJECT_ROOT / \"data\" / \"bronze\"\n",
  "\n",
  "REMEDIED_RAW_ROOT = PROJECT_ROOT / \"data\" / \"remediated\" / \"raw\"\n",
  "REMEDIED_BRONZE_ROOT = PROJECT_ROOT / \"data\" / \"remediated\" / \"bronze\"\n",
  "\n",
  "REPORT_ROOT = PROJECT_ROOT / \"reports\" / \"validation\"\n",
  "LOG_ROOT = PROJECT_ROOT / \"logs\"\n",
  "\n",

```

14. Data Quality JSON Preview

```

{
  "run_id": "20260429T111342Z",
  "run_ts": "2026-04-29T11:13:42.777701+00:00",
  "project_root": "C:\\Users\\barath\\recomart-pipeline",
  "initial_validation": {
    "run_id": "20260429T111342Z",
    "run_ts": "2026-04-29T11:13:42.777701+00:00",
    "phase": "initial",
    "project_root": "C:\\Users\\barath\\recomart-pipeline",
    "raw_root": "C:\\Users\\barath\\recomart-pipeline\\data\\raw",
    "bronze_root": "C:\\Users\\barath\\recomart-pipeline\\data\\bronze",
    "discovered_files": {
      "events_csv": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-29\\load_hour=11\\events.csv",
      "category_tree_csv": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-29\\load_hour=11\\category_tree.csv",
      "item_properties_part1_csv": "C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-29\\load_hour=11\\item_properties_part1.csv",
      "item_properties_part2_csv": "C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-29\\load_hour=11\\item_properties_part2.csv",
      "products_raw_json": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-29\\load_hour=11\\products_raw.json",
      "categories_raw_json": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-29\\load_hour=11\\categories_raw.json",
      "products_parquet": "C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\products.parquet",
      "categories_parquet": "C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\categories.parquet"
    },
    "overall_status": "FAIL",
    "issues_file": "C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_issues_initial_20260429T111342Z.csv",
    "summary_file": "C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_summary_initial_20260429T111342Z.csv",
    "datasets": [
      {
        "phase": "initial",
        "dataset": "retailrocket_events",
        "rows": 2756101,
        "columns": 5,
        "errors": 0,
        "warnings": 2734104,
        "status": "PASS"
      },
      {
        "phase": "initial",
        "dataset": "retailrocket_category_tree",
        "rows": 1669,

```

```

    "columns": 2,
    "errors": 0,
    "warnings": 25,
    "status": "PASS"
  },
  {
    "phase": "initial",
    "dataset": "retailrocket_item_properties",
    "rows": 20275902,
    "columns": 4,
    "errors": 0,
    "warnings": 0,
    "status": "PASS"
  },
  {
    "phase": "initial",
    "dataset": "dummyjson_products_raw",
    "rows": 194,
    "columns": 27,
    "errors": 0,
    "warnings": 92,
    "status": "PASS"
  },
  {
    "phase": "initial",
    "dataset": "dummyjson_categories_raw",
    "rows": 24,
    "columns": 3,
    "errors": 1,
    "warnings": 0,
    "status": "FAIL"
  },
},

```

15. Fix Log Preview

```

dataset,action,rows_before,rows_after,details,file_path
retailrocket_events,remove_invalid_event_values,2756101,2756101,Kept only allowed event
types,C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=11\events.csv
retailrocket_events,drop_invalid_timestamps,2756101,2756101,Removed rows with invalid
timestamp,C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=11\events.csv
retailrocket_events,drop_duplicates,2756101,2755641,Removed duplicate event
rows,C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=11\events.csv
retailrocket_events,drop_missing_required_rows,2755641,2755641,Dropped rows missing required
fields,C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=11\events.csv
retailrocket_category_tree,drop_missing_categoryid,1669,1669,Dropped rows with missing
categoryid,C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=11\category_tree.csv
retailrocket_category_tree,drop_duplicates,1669,1669,Removed duplicate category
rows,C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=11\category_tree.csv
retailrocket_item_properties,drop_missing_required_rows,20275902,20275902,Dropped rows with
invalid required values,C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=11\item_properties_part1.csv
retailrocket_item_properties,drop_duplicates,20275902,20275902,Removed duplicate property
rows,C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=11\item_properties_part1.csv
dummyjson_products_raw,drop_missing_required_rows,194,194,Dropped rows missing
id/title/category/price,C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=11\products_raw.json
dummyjson_products_raw,drop_duplicate_ids,194,194,Kept first product per
id,C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=11\products_raw.json
dummyjson_products_raw,fix_negative_prices,194,194,Clipped 0 negative prices to
0,C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=11\products_raw.json
dummyjson_products_raw,clip_ratings,194,194,Clipped 0 rating values into
1-5,C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=11\products_raw.json
dummyjson_products_raw,fix_negative_stock,194,194,Clipped 0 stock values to
0,C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=11\products_raw.json
dummyjson_categories_raw,drop_missing_category,24,0,Dropped null
categories,C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=11\categories_raw.json
dummyjson_categories_raw,drop_blank_category,0,0,Dropped blank
categories,C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=11\categories_raw.json
dummyjson_categories_raw,drop_duplicates,0,0,Removed duplicate
categories,C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=11\categories_raw.json
products_parquet,add_missing_bronze_metadata,194,194,Ensured bronze metadata columns
exist,C:\Users\barath\recomart-pipeline\data\bronze\dummyjson\products.parquet
categories_parquet,add_missing_bronze_metadata,24,24,Ensured bronze metadata columns

```

```
exist,C:\Users\barath\recomart-pipeline\data\bronze\dummyjson\categories.parquet
```

16. Conclusion

This document captures the project evidence for the Data Preparation stage, including cleaning and preprocessing logic, EDA plots, prepared dataset output, and supporting logs needed for a submission-ready PDF deliverable.